PROBLEM STATEMENT NO. 183 · CATEGORY: MODEL MAKING

Multi-Document RAG with Citations

An AI-powered system for reliable, source-verified question answering across multiple documents

PRESENTED BY

Honey Soni (24, ADT23SOCB0444)
Roshan Warade (35, ADT23SOCB0883)
Virendra Mahajan (41, ADT23SOCB1322)
Poornima Rayavarapu (31, ADT23SOCB0727)
Sujal Jadhavar (37 , ADT23SOCB1167)

THE PROBLEM

Answers live in many documents at once

Information that answers a single question is rarely in one file. It is spread across policies, reports, manuals and research papers.

Policies and guidelines

Rules, eligibility criteria, approval routes

Academic and research documents

Papers, syllabi, standards, technical notes

Reports and manuals

Annual figures, procedures, operating handbooks

WHY THIS IS HARD

Manual search is slow

Reading four documents to find one line

Relevant details get missed

The answer may sit in the file you skipped

AI can answer without evidence

Fluent, confident and sometimes invented

The source cannot be checked

No way to confirm where an answer came from

PROPOSED SOLUTION

A Multi-Document RAG system with citations

1. **Accept multiple documents as one knowledge source**

   PDFs, reports, notes — mixed formats in a single corpus

2. **Understand and process document content**

   PDF, Word and text files, with OCR for scans and photos

3. **Retrieve only what is relevant to the query**

   Hybrid semantic and keyword search, then reranking

4. **Generate the answer from that retrieved context**

   The model is restricted to the evidence it was given

5. **Attach document and page citations**

   Every claim carries a [Doc, Page] marker to its source

CORE IDEA

Retrieve first.
Generate second.

The relevant passages are found before the model writes anything. It never answers from memory — it answers from the text it was just shown.

Built-in refusal

When the documents do not hold the answer, it says so. A general-knowledge reply is always flagged.

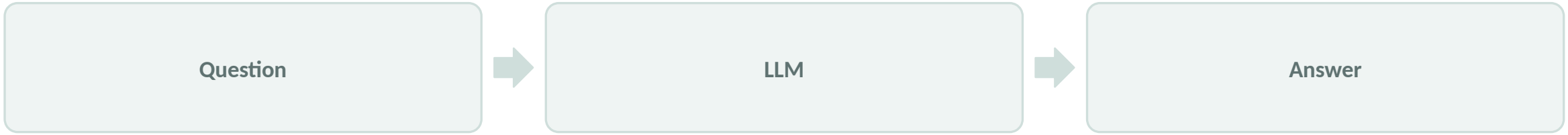
UNDERSTANDING RAG

Retrieval-Augmented Generation

Instead of letting the model answer directly from what it memorised during training, RAG searches the provided documents first and answers only from what it finds.

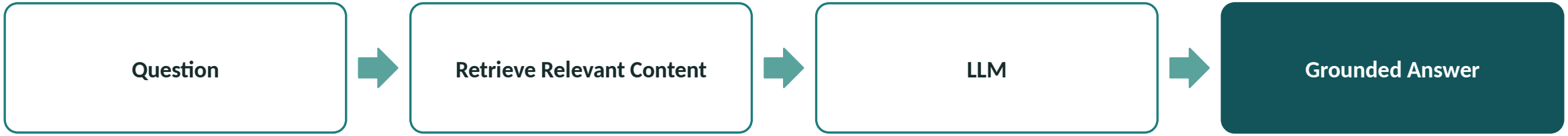
TRADITIONAL APPROACH

answers from memory — no source to check



OUR RAG APPROACH

answers from the documents — every claim traceable



Main benefit: the answer is generated from information retrieved out of the provided documents, so every claim can be checked against the exact page it came from.

SYSTEM METHODOLOGY

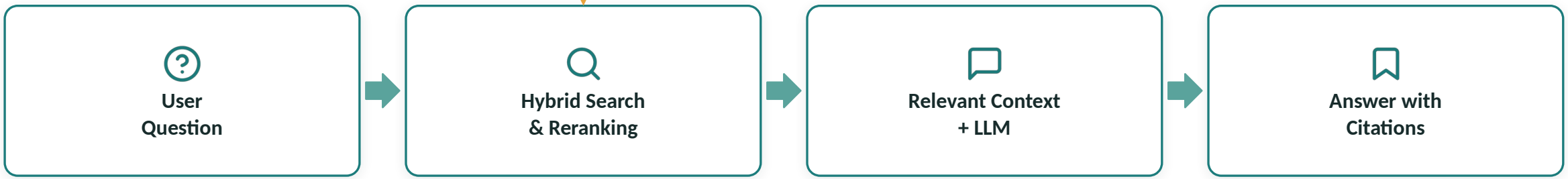
How the system works

DOCUMENT PROCESSING PIPELINE



the knowledge base built above is what every query searches

QUESTION ANSWERING PIPELINE



Pipeline 1 runs once
when a document set is added or changed

Pipeline 2 runs every time
a question is asked, answered in about 8 seconds

Citations come from metadata
the document and page stored beside each vector

STAGE ONE

From documents to a searchable knowledge base

Text Extraction

1

pdfplumber reads text and tables page by page. Scanned pages and photos go through Tesseract OCR in English, Hindi and Marathi.

Chunking

2

Each page is split into passages of about 800 characters with a 150-character overlap, breaking at paragraphs and sentences.

Embeddings

3

nomic-embed-text turns each passage into a 768-dimension vector, so passages with related meaning sit close together.

Vector Storage

4

Each vector is saved with its document name, page and chunk index, next to a BM25 keyword index. Both are kept on disk.

Result: one searchable knowledge base built from many documents — and every passage in it still knows exactly where it came from.

STAGE TWO

From a question to a cited answer

1. **User asks a question**

   In plain language, as they would ask a person

2. **Hybrid search**

   Vector similarity and BM25 keyword scores, merged by rank fusion

3. **Reranking**

   A cross-encoder rescores 20 candidates and keeps the best 4

4. **Answer generation**

   Only those passages are given to the LLM, with instructions to use nothing else

5. **Citations attached**

   Each claim is tagged [Doc, Page]; one click opens the exact passage

WORKED EXAMPLE

“What are the eligibility requirements?”

RETRIEVED PASSAGES

- S1 admissions-policy.pdf · p. 3
- S2 programme-handbook.pdf · p. 11
- S3 fee-circular.pdf · p. 2

Answer drawn from 3 documents

cited as S1, S2 and S3 inside the text

INPUT AND EXPECTED OUTPUT

What goes in, what comes out

INPUT

- Document 1 · policy
- Document 2 · handbook
- Document 3 · circular

USER QUESTION

“What are the placement eligibility requirements?”

EXPECTED OUTPUT

GENERATED ANSWER

A student must have cleared all backlogs and hold an aggregate of 60 percent to register for placement.

SUPPORTING CITATIONS

- S1 admissions-policy.pdf · p. 3
- S2 programme-handbook.pdf · p. 11

Answer + Evidence + Verification

TOOLS AND IMPLEMENTATION

Technology stack

Backend	Python 3.13 with FastAPI
Document processing	pdfplumber, pypdf, python-docx, Tesseract OCR
Chunking	800-character passages, 150 overlap, page-aware
Embeddings	nomic-embed-text, 768 dimensions (local)
Vector store	NumPy vectors + BM25 index, saved to disk
Retrieval	Cosine + BM25, rank fusion, MiniLM reranker
Generation	Qwen 2.5 7B on Ollama (Llama 3.1 8B in jury)
Interface	React 18, TypeScript, Vite, Tailwind CSS
Evaluation	Claim audit, Hit@k and MRR self-tuning

FINAL IMPLEMENTATION

A web application where a user can:

- ✓ Upload multiple documents
- ✓ Ask questions in natural language
- ✓ Get streamed, cited answers
- ✓ Open the passage behind a citation



Runs entirely offline

The language model, embeddings, reranker and speech-to-text all run on the local machine. No API keys, and no document ever leaves the laptop.

BEYOND QUESTION ANSWERING

Features built on top of the pipeline

Answer Audit

1

Each answer is split into claims and checked against its sources. It gets a faithfulness score, and unsupported claims are removed.

Model Jury

2

Qwen 2.5 7B and Llama 3.1 8B answer from the same passages, and a judge model reports where the two agree or differ.

Document Insights

3

After upload, each file gets a summary, FAQ, quiz, timeline and knowledge graph, and is checked for clashes with other files.

Voice and Telegram

4

Questions can be spoken in English, Hindi or Marathi and transcribed locally by Whisper, or sent through a Telegram bot.

Self-tuning: user feedback builds a test set every night, and retrieval settings change only when Hit@k or MRR clearly improves.

OUTCOME

What the system delivers

Faster retrieval

Seconds instead of manual reading

Across documents

One question, the whole corpus

Grounded answers

Generated from retrieved text only

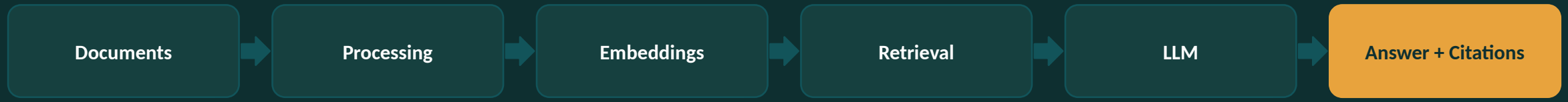
Source-supported

Document and page on every claim

Verifiable

Answers audited claim by claim

THE COMPLETE JOURNEY



FINAL GOAL

To build a reliable GenAI system that helps users obtain accurate and verifiable information from multiple documents.